Your Paper Title Goes Here

First Author* Second Author†

October 25, 2026

Organization. Section 2 sets up notation, Section 3 presents our approach, **??** reports results, and Section 4 concludes. Omitted proofs and additional details appear in the appendix.

1 Related Work

Group prior work by theme and explain how yours differs.

2 Preliminaries

The algorithm we wish to make takes in a (wlog injective) function $f : [n] \to \mathbb{N}$ [1] and returns the maximum fraction of points in the longest increasing subsequence. The maximum fraction of points in the longest increasing subsequence is defined as the largest number of elements in $[n]$ that can be kept such that f is monotonic, divided by n. [2].

The imput f can be thought of being embedded in the 2d cartesian plane where the x coordinate (subsequently called 1 coordinate) is the input for f, and the y coordinate (subsequently called the 2 coordinate) is the output for f. Any interval of 1 coordinates is called an index interval and any interval of 2 coordinates is called a value interval.

A box $\mathcal{B}$ is formed by the cartesian product of a connected index and connected value interval. For simplicity, all boxes referenced here will have half integer coordinates. [3] There is a bounding box $\boldsymbol{B}$ that contains all of the points of f, i.e. $\boldsymbol{B} \triangleq [-0.5, n + 0.5] \times [-0.5, \max(range(f)) + 0.5]$.

For any box $\mathcal{B}$, we will use $1(\mathcal{B})$ to represent the index interval of $\mathcal{B}$ and $2(B)$ to represent the value interval of $\mathcal{B}$.

A grid $G(\mathcal{B})$ is a cartesian product of a subset of $1(\mathcal{B}) \times 2(\mathcal{B})$ containing only half integers.

$$G : \mathcal{B} \mapsto X \times Y \subseteq 1(\mathcal{B}) \times 2(\mathcal{B}) \tag{1}$$

It may seem like this set is not a grid, but one can always construct a grid with these subsets by drawing vertical lines at each element in the first coordinate, and horizontal lines at each element in the second coordinate.

*Affiliation. Email: `first@example.edu`

†Affiliation. Email: `second@example.edu`

[1] Where $[n]$ is the set $\{0, 1, 2, \ldots n - 1\}$.

[2] alternatively, it is n minus the number of elements that must be removed such that f is monotonic, all divided by n

[3] This simplification means the corners of the boxes never intersect with points of f

Given a grid $G(\mathcal{B}_1)$, one can construct/induce a grid $G(\mathcal{B}_2, \mathcal{B}_1)$ on any $\mathcal{B}_2 \subseteq \mathcal{B}_1$ by considering the projection map of the gridpoints onto $\mathcal{B}_2$. i.e. remove all the gridpoints in $G(\mathcal{B}_1)$ that are not in the index/value intervals of $\mathcal{B}_2$.

Note that $g(\mathcal{B})$ induces $O(n^4)$ subboxes of $\mathcal{B}$ [4] formed by picking 4 elements in $g(\mathcal{B})$. We will call a box induced by $g(\mathcal{B})$ a gridbox. [5] A minimal gridbox is a box whose index interval is bounded by two consecutive X elements, and whose value interval is bounded by two consecutive Y elements, i.e. a minimal gridbox contains no strictly smaller non-degenerate gridbox. A minimal gridwidth is an index interval of a minimal gridbox.

A gridchain $C(G(\mathcal{B}))$ is a union of gridboxes with several properties:

1. each gridbox has minimal gridwidth.
2. the union of the gridboxes' 1 intervals fully spans $1(\mathcal{B})$.
3. the union of the gridboxes' 2 intervals fully spans $2(\mathcal{B})$.
4. no two gridboxes' 1 intervals intersect.
5. no two gridboxes' 2 intervals intersect.
6. the corners of each gridbox must be equal to either
   (a) the bottom left of $\mathcal{B}$
   (b) the top right of $\mathcal{B}$
   (c) another gridbox in the gridchain's top right or bottom left corner.

We will also use $st(\mathcal{B},\mathcal{U})$ to represent the points in $\mathcal{U}$ with 1 index in $1(B)$.

There are several sub-procedures we will make use of repeatedly, so we will frame these procedures as oracles. The constants will be picked such that each oracle call can be implemented in polylog time and the proofs for the existence of these oracles can be found in the appendix. [6]

We will use the hat symbol (ex: $\hat{w}(\boldsymbol{B})$) to represent the true values and no hats (ex: $w(\boldsymbol{B})$) to represent estimate values from the oracles below. Repeated calls with the same arguments return the same answer.

Theorem 2.1. *Given any $\delta > 0$, There exists a width oracle $w : \mathcal{B} \times \mathcal{U} \to \mathbb{N}$ that maps two boxes $\mathcal{B} \subseteq \mathcal{U}$ to $\mathbb{N}$ and reports the number of points in $\mathcal{B}$'s 1 interval that are contained in $\mathcal{U}$ with error at most $\delta\hat{w}(\mathcal{U}, \boldsymbol{B})$*[7]

Theorem 2.2. *Given any $\delta > 0$, There exists a size oracle $s : \mathcal{B} \to \mathbb{N}$ that maps a box $\mathcal{B}$ to $\mathbb{N}$ and reports the number of points in $\mathcal{B}$'s with error at most $\delta\hat{w}(\mathcal{B}, \boldsymbol{B})$*

Theorem 2.3. *given any $\alpha > 0, \omega > 0, \eta \in [0, 1]$, there exists a grid oracle `BuildGrid`, $g : \mathcal{B} \to G(\mathcal{B})$ that maps a box $\mathcal{B}$ to a grid $G(\mathcal{B})$ with the following properties:*

1. *If $w(\mathcal{B}, \boldsymbol{B}) < \omega$, read all of the elements and return a grid with exactly at most one point in each minimal gridbox.*
2. *If $w(\mathcal{B}, \boldsymbol{B}) > \omega$,*

[4] subboxes are formed this way, each subbox is the same as choosing two index points and two value points that bound the new subbox which is $\binom{n}{2} \times \binom{n}{2} = O(n^4)$

[5] Note, picking the same starting and ending index/value intervals are allowed in constructing gridboxes. We will call these gridboxes "degenerate".

[6] the wording here is not very good. i also need to include how the runtime is ok despite being able to tune parameters δ and α

[7] where do i add the "whp" clause here?

Algorithm 1 Example Algorithm

Require: Array $A[1..n]$
Ensure: Output value

```
s ← 0
for i ← 1 to n do
    if A[i] > s then
        s ← A[i]
    end if
end for
return s
```

3 Main Results

Theorem 3.1. *There is an algorithm that solves the problem in $O(n \log n)$ time.*

Proof. Proof sketch here. (With `theoremrep`, use `\begin{toappendix}` or `\begin{proofrep}` to move the full proof to the appendix.) □

Lemma 3.2. *A helper lemma whose proof is deferred.*

4 Conclusion

Summarize findings, limitations, and future directions.

Acknowledgments

Thank collaborators and funding sources.

References

A Omitted Proofs

Proof of Theorem 3.1. Full, detailed proof of the main theorem. □

B Additional Experimental Details

B.1 Setup

Hardware, software versions, hyperparameters, datasets.

B.2 Additional Results

C Supplementary Material

Extra derivations, pseudocode, or tables that don't fit in the main body.

Table 1: Extended results.

Method	A	B	C
Baseline	0.71	12.3	5.1
Ours	0.84	9.8	4.2

A Proofs for Section 2 (Preliminaries)

Theorem 2.1. *Given any $\delta > 0$, There exists a width oracle $w : \mathcal{B} \times \mathcal{U} \to \mathbb{N}$ that maps two boxes $\mathcal{B} \subseteq \mathcal{U}$ to $\mathbb{N}$ and reports the number of points in $\mathcal{B}$'s 1 interval that are contained in $\mathcal{U}$ with error at most $\delta\hat{w}(\mathcal{U}, \boldsymbol{B})$*[8]

Proof. □

Theorem 2.2. *Given any $\delta > 0$, There exists a size oracle $s : \mathcal{B} \to \mathbb{N}$ that maps a box $\mathcal{B}$ to $\mathbb{N}$ and reports the number of points in $\mathcal{B}$'s with error at most $\delta\hat{w}(\mathcal{B}, \boldsymbol{B})$*

Proof. □

Theorem 2.3. *given any $\alpha > 0, \omega > 0, \eta \in [0, 1]$, there exists a grid oracle `BuildGrid`, $g : \mathcal{B} \to G(\mathcal{B})$ that maps a box $\mathcal{B}$ to a grid $G(\mathcal{B})$ with the following properties:*

1. *If $w(\mathcal{B}, \boldsymbol{B}) < \omega$, read all of the elements and return a grid with exactly at most one point in each minimal gridbox.*
2. *If $w(\mathcal{B}, \boldsymbol{B}) > \omega$,*

B Proofs for Section 3 (Main Results)

Lemma 3.2. *A helper lemma whose proof is deferred.*

Proof. This full proof is automatically moved to the appendix by `apxproof`. □

[8] where do i add the "whp" clause here?